

Отчёт по лабораторной работе №5

**Дискреционное разграничение прав в Linux. Исследование
влияния дополнительных атрибутов**

Шаханеоядж Хаоладар

Содержание

1	Цель работы	5
2	Выполнение лабораторной работы	6
2.1	Подготовка	6
2.2	Изучение механики SetUID	7
2.3	Исследование Sticky-бита	15
3	Выводы	18
	Список литературы	19

Список иллюстраций

2.1	подготовка к работе	7
2.2	программа simpleid	8
2.3	результат программы simpleid	9
2.4	программа simpleid2	10
2.5	результат программы simpleid2	12
2.6	программа readfile	13
2.7	результат программы readfile	14
2.8	исследование Sticky-бита	17

Список таблиц

1 Цель работы

Изучение механизмов изменения идентификаторов, применения SetUID и Sticky-битов. Получение практических навыков работы в консоли с дополнительными атрибутами. Рассмотрение работы механизма смены идентификатора процессов пользователей, а также влияние бита Sticky на запись и удаление файлов.

2 Выполнение лабораторной работы

2.1 Подготовка

1. Для выполнения части заданий требуются средства разработки приложений. Проверили наличие установленного компилятора gcc командой `gcc -v`: компилятор обнаружен.
2. Чтобы система защиты SELinux не мешала выполнению заданий работы, отключили систему запретов до очередной перезагрузки системы командой `setenforce 0`:
3. Команда `getenforce` вывела `Permissive`:

```

guest@haoladar:~$ gcc -v
Using built-in specs.
COLLECT_GCC=gcc
COLLECT_LTO_WRAPPER=/usr/libexec/gcc/x86_64-redhat-linux/14/lto-wrapper
OFFLOAD_TARGET_NAMES=nvptx-none:amdgc-n-amdhsa
OFFLOAD_TARGET_DEFAULT=1
Target: x86_64-redhat-linux
Configured with: ../configure --enable-bootstrap --enable-languages=c,c++,fortran,lto --pre
fix=/usr --mandir=/usr/share/man --infodir=/usr/share/info --with-bugurl=https://bugs.rocky
linux.org/ --enable-shared --enable-threads=posix --enable-checking=release --enable-multil
ib --with-system-zlib --enable-__cxa_atexit --disable-libunwind-exceptions --enable-gnu-uni
que-object --enable-linker-build-id --with-gcc-major-version-only --enable-libstdcxx-backtr
ace --with-libstdcxx-zoneinfo=/usr/share/zoneinfo --with-linker-hash-style=gnu --enable-plu
gin --enable-initfini-array --without-isl --enable-offload-targets=nvptx-none,amdgc-n-amdhsa
--enable-offload-defaulted --without-cuda-driver --enable-gnu-indirect-function --enable-c
et --with-tune=generic --with-arch_64=x86-64-v3 --with-arch_32=x86-64 --build=x86_64-redhat
-linux --with-build-config=bootstrap-lto --enable-link-serialization=1 --enable-host-pie --
enable-host-bind-now
Thread model: posix
Supported LTO compression algorithms: zlib zstd
gcc version 14.3.1 20250617 (Red Hat 14.3.1-2) (GCC)
guest@haoladar:~$ su
Password:
root@haoladar:/home/guest# setenforce 0
root@haoladar:/home/guest# getenforce
Permissive
root@haoladar:/home/guest# █

```

Рисунок 2.1: подготовка к работе

2.2 Изучение механики SetUID

1. Вошли в систему от имени пользователя guest.
2. Написали программу simpleid.c.

```
#include <sys/types.h>
#include <unistd.h>
#include <stdio.h>

int main()
{
    uid_t uid = geteuid();
    gid_t gid = getegid();
    printf("uid=%d, gid=%d\n", uid, gid);
    return 0;
}
```

Рисунок 2.2: программа simpleid

3. Скомпилировали программу и убедились, что файл программы создан:
gcc simpleid.c -o simpleid
4. Выполнили программу simpleid командой ./simpleid
5. Выполнили системную программу id с помощью команды id. uid и gid совпадает в обеих программах


```
guest@haoladar:~/lab5$  
guest@haoladar:~/lab5$ gcc simpleid.c  
guest@haoladar:~/lab5$ gcc simpleid.c -o simpleid  
guest@haoladar:~/lab5$ ./simpleid  
uid=1002, gid=1002  
guest@haoladar:~/lab5$ id  
uid=1002(guest) gid=1002(guest) groups=1002(guest) context=unconfined_u:unconfined_r:unconf  
ined_t:s0-s0:c0.c1023  
guest@haoladar:~/lab5$
```

Рисунок 2.3: результат программы simpleid

6. Усложнили программу, добавив вывод действительных идентификаторов.



```
Open ▾  simpleid2.c  
~/lab5  
  
#include <sys/types.h>  
#include <unistd.h>  
#include <stdio.h>  
  
int main()  
{  
    uid_t e_uid = geteuid();  
    gid_t e_gid = getegid();  
    uid_t real_uid = getuid();  
    gid_t real_gid = getgid();  
    printf("e_uid=%d, e_gid=%d\n", e_uid, e_gid);  
    printf("real_uid=%d, real_gid=%d\n", real_uid, real_gid);  
    return 0;  
}
```

Рисунок 2.4: программа simpleid2

7. Скомпилировали и запустили simpleid2.c:

```
gcc simpleid2.c -o simpleid2  
./simpleid2
```

8. От имени суперпользователя выполнили команды:

```
chown root:guest /home/guest/simpleid2  
chmod u+s /home/guest/simpleid2
```

9. Использовали su для повышения прав до суперпользователя

10. Выполнили проверку правильности установки новых атрибутов и смены владельца файла simpleid2:

```
ls -l simpleid2
```

11. Запустили simpleid2 и id:

```
./simpleid2
```

```
id
```

Результат выполнения программ теперь немного отличается

12. Проделали тоже самое относительно SetGID-бита.

```

guest@haoladar:~/lab5$ gcc simpleid2.c
guest@haoladar:~/lab5$ gcc simpleid2.c -o simpleid2
guest@haoladar:~/lab5$ ./simpleid2
e_uid=1002, e_gid=1002
real_uid=1002, real_gid=1002
guest@haoladar:~/lab5$ su
Password:
root@haoladar:/home/guest/lab5# chown root:guest simpleid2
root@haoladar:/home/guest/lab5# chmod u+s simpleid2
root@haoladar:/home/guest/lab5# ./simpleid2
e_uid=0, e_gid=0
real_uid=0, real_gid=0
root@haoladar:/home/guest/lab5# id
uid=0(root) gid=0(root) groups=0(root) context=unconfined_u:unconfined_r:unconfined_t:s0-s0
:c0.c1023
root@haoladar:/home/guest/lab5# chmod g+s simpleid2
root@haoladar:/home/guest/lab5# ./simpleid2
e_uid=0, e_gid=1002
real_uid=0, real_gid=0
root@haoladar:/home/guest/lab5#
exit
guest@haoladar:~/lab5$ ./simpleid2
e_uid=0, e_gid=1002
real_uid=1002, real_gid=1002
guest@haoladar:~/lab5$

```

Рисунок 2.5: результат программы simpleid2

13. Написали программу readfile.c

The image shows a code editor window with the title 'readfile.c' and a path '~ /lab5'. The code is written in C and includes several standard headers: <stdio.h>, <sys/stat.h>, <sys/types.h>, <unistd.h>, and <fcntl.h>. The main function takes two arguments, argc and argv. It declares a buffer of 16 unsigned bytes, a size_t variable for bytes_read, and an int variable i. It opens the file specified in argv[1] in read-only mode. A 'do' loop contains a 'read' call, a 'for' loop to print each byte of the buffer, and a 'while' loop that continues as long as bytes_read is equal to the buffer size. After the loop, it closes the file and returns 0.

```
#include <stdio.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <unistd.h>
#include <fcntl.h>

int main(int argc, char* argv[])
{
    unsigned char buffer[16];
    size_t bytes_read;
    int i;

    int fd=open(argv[1], O_RDONLY);
    do
    {
        bytes_read=read(fd, buffer, sizeof(buffer));
        for (i=0; i<bytes_read; ++i)
            printf("%c", buffer[i]);
    }
    while (bytes_read == (buffer));
    close (fd);
    return 0;
}
```

Рисунок 2.6: программа readfile

14. Откомпилировали её.

```
gcc readfile.c -o readfile
```

15. Сменили владельца у файла readfile.c и изменили права так, чтобы только суперпользователь (root) мог прочитать его, а guest не мог.

```
chown root:guest /home/guest/readfile.c
```

```
chmod 700 /home/guest/readfile.c
```

16. Проверили, что пользователь guest не может прочитать файл readfile.c.
17. Сменили у программы readfile владельца и установили SetU'D-бит.
18. Проверили, может ли программа readfile прочитать файл readfile.c
19. Проверили, может ли программа readfile прочитать файл /etc/shadow

```
guest@haoladar:~/lab5$ gcc readfile.c
readfile.c: In function 'main':
readfile.c:20:19: warning: comparison between pointer and integer
   20 | while (bytes_read == (buffer));
      |                   ^~

guest@haoladar:~/lab5$ gcc readfile.c -o readfile
readfile.c: In function 'main':
readfile.c:20:19: warning: comparison between pointer and integer
   20 | while (bytes_read == (buffer));
      |                   ^~

guest@haoladar:~/lab5$ su
Password:
root@haoladar:/home/guest/lab5# chown root:root readfile
root@haoladar:/home/guest/lab5# chmod -rwx readfile.c
root@haoladar:/home/guest/lab5# chmod u+s readfile
root@haoladar:/home/guest/lab5#
exit
guest@haoladar:~/lab5$ cat readfile.c
cat: readfile.c: Permission denied
guest@haoladar:~/lab5$ ./readfile readfile.c
#include <stdio.h>
guest@haoladar:~/lab5$
guest@haoladar:~/lab5$ ./readfile /etc/shadow
root:$y$j9T$Vr4/guest@haoladar:~/lab5$
guest@haoladar:~/lab5$
```

Рисунок 2.7: результат программы readfile

2.3 Исследование Sticky-бита

1. Выяснили, установлен ли атрибут Sticky на директории /tmp:

```
ls -l / | grep tmp
```

2. От имени пользователя guest создали файл file01.txt в директории /tmp со словом test:

```
echo "test" > /tmp/file01.txt
```

3. Просмотрели атрибуты у только что созданного файла и разрешили чтение и запись для категории пользователей «все остальные»:

```
ls -l /tmp/file01.txt  
chmod o+rw /tmp/file01.txt  
ls -l /tmp/file01.txt
```

Первоначально все группы имели право на чтение, а запись могли осуществлять все, кроме «остальных пользователей».

4. От пользователя (не являющегося владельцем) попробовали прочитать файл /file01.txt:

```
cat /file01.txt
```

5. От пользователя попробовали дозаписать в файл /file01.txt слово test3 командой:

```
echo "test2" >> /file01.txt
```

6. Проверили содержимое файла командой:

```
cat /file01.txt
```

В файле теперь записано:

Test

Test2

7. От пользователя попробовали записать в файл /tmp/file01.txt слово test4, стерев при этом всю имеющуюся в файле информацию командой. Для этого воспользовалась командой `echo «test3» > /tmp/file01.txt`

8. Проверили содержимое файла командой

```
cat /tmp/file01.txt
```

9. От пользователя попробовали удалить файл /tmp/file01.txt командой `rm /tmp/file01.txt`, однако получила отказ.
10. От суперпользователя командой выполнили команду, снимающую атрибут t (Sticky-бит) с директории /tmp:

```
chmod -t /tmp
```

Покинули режим суперпользователя командой `exit`.

11. От пользователя проверили, что атрибута t у директории /tmp нет:

```
ls -l / | grep tmp
```

12. Повторили предыдущие шаги. Получилось удалить файл
13. Удалось удалить файл от имени пользователя, не являющегося его владельцем.
14. Повысили свои права до суперпользователя и вернули атрибут t на директорию /tmp :


```
su
```

```
chmod +t /tmp
```

```
exit
```

```
guest@haoladar: /tmp$  
guest@haoladar:~/lab5$ echo "test" >> /tmp/file01.txt  
guest@haoladar:~/lab5$ chmod g+rw /tmp/file01.txt  
guest@haoladar:~/lab5$ su guest2  
Password:  
guest2@haoladar:/home/guest/lab5$ cd /tmp  
guest2@haoladar:/tmp$ cat file01.txt  
test  
guest2@haoladar:/tmp$ echo "test2" >> /tmp/file01.txt  
guest2@haoladar:/tmp$ cat file01.txt  
test  
test2  
guest2@haoladar:/tmp$ echo "test3" > /tmp/file01.txt  
guest2@haoladar:/tmp$ rm file01.txt  
rm: cannot remove 'file01.txt': Operation not permitted  
guest2@haoladar:/tmp$ su  
Password:  
root@haoladar:/tmp# chmod -t /tmp  
root@haoladar:/tmp#  
exit  
guest2@haoladar:/tmp$ cd /tmp  
guest2@haoladar:/tmp$ echo "test2" >> /tmp/file01.txt  
guest2@haoladar:/tmp$ rm file01.txt  
guest2@haoladar:/tmp$
```

Рисунок 2.8: исследование Sticky-бита

3 Выводы

Изучили механизмы изменения идентификаторов, применения SetUID- и Sticky-битов. Получили практические навыки работы в консоли с дополнительными атрибутами. Также мы рассмотрели работу механизма смены идентификатора процессов пользователей и влияние бита Sticky на запись и удаление файлов.

Список литературы

1. КОМАНДА CHATTR В LINUX
2. chattr